MICROPROCESSORS

PAGE NO.
DATE: 1-7

INTRO

Bonus

→ i9 can understand 8086. (possible)

→ can run older programs in Virtual Simulated Environment (dosbox).
[Disk operating System]

→ CPU (Central/General Processing Unit) —normal work→

↓ ↑

IOP

Input/output Processor?

→ PPIc → Peripheral Programmable IC.

→ Thread = Parallelism at user level

→ OpenCL or CUDA (for NVIDIA)
Open Computing Lang. Compute Unified D... Architechture.

→ ARM → RISC ; 85 or 86 → CISC (no pure RISC/cisc exist today).
32 bit 8 bit 16 bit
i7+ → 64 bit

→ Python is a script. like BASIC.

→ Assembler. + Assembly lang.

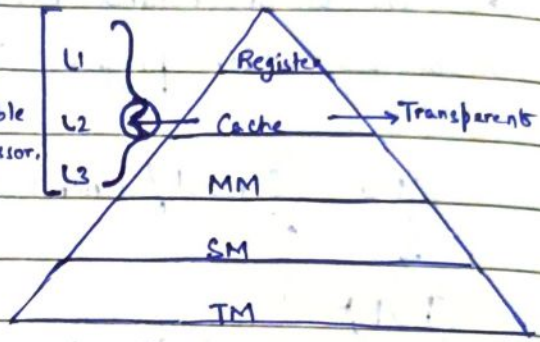
[Book → Douglous Hall and Donker]

PAGE NO.
DATE 27/July/

Memory

→ 2nd most important thing. followed by processor and leading interfacing.

- At the very least Register and Cache are there in processor. L1, L2 and L3 are usually there in processor.



- Stored Prg Mem → write program to run — — and once executed it will get cleared from it. Therefore, we save it.

- Prg → Code, Data } earlier both were in same. Then code and data were separated in different memory.

  - Pristen/Von neumann Architecture. → can't read code once writing data. Processor goes idle. (single pointer)
  - Harvard Architecture. → writing data of prev. instruction and exe. data of next instruction is possible. (2 pointers).

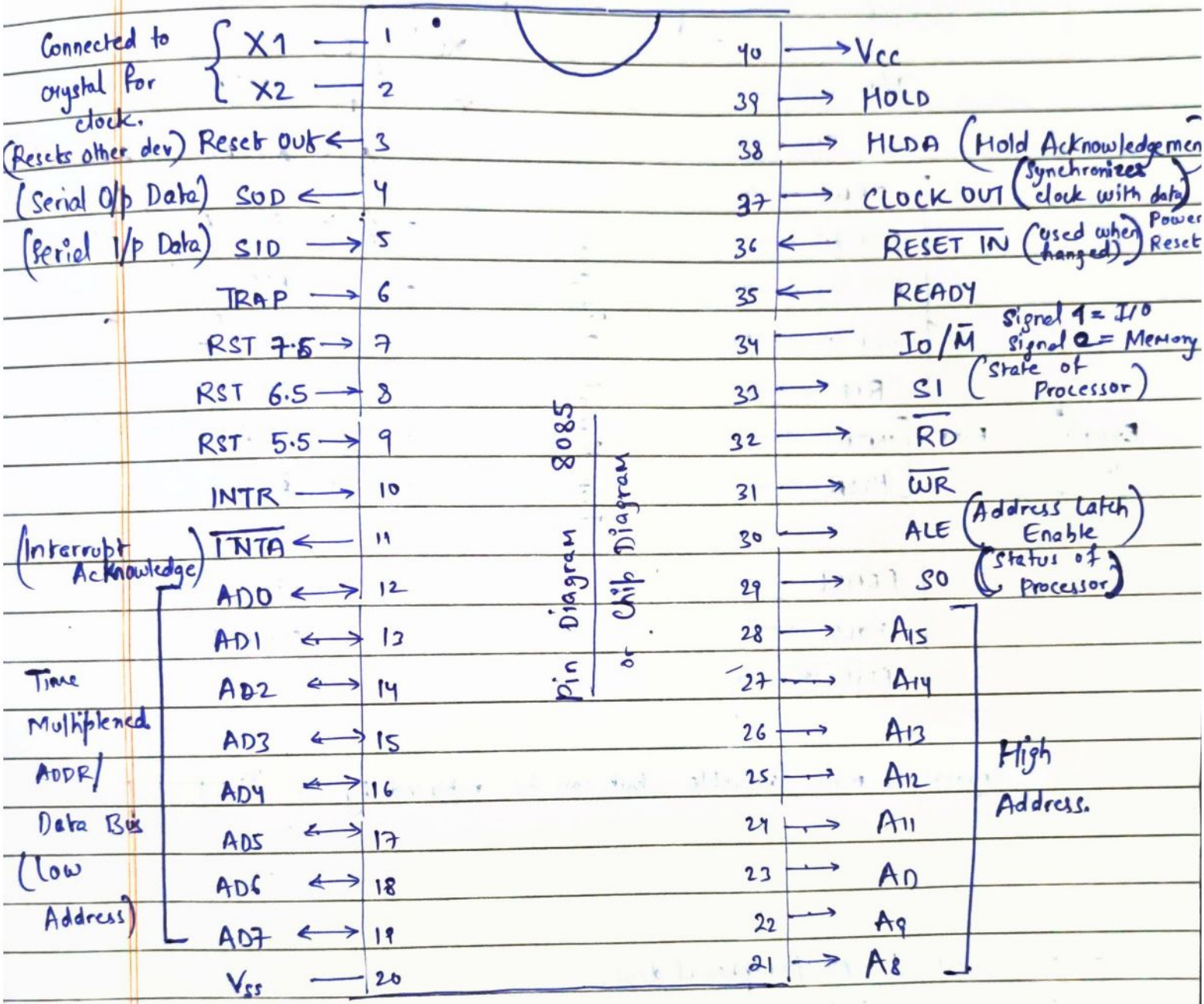
- In Harvard we have L1I → Instruction, ~~Indirect~~ and L1D → ~~Direct~~ Data → 2 kinds of cache.

  L2 and L3 → unified/hybrid memory.

→ See Interfacing

PAGE NO.
DATE: 28 'July'

8085



or receive

→ Can be used to send one bit at a time. (So serial usage is possible)

→ 5 types of external interrupts from TRAP to INTR. (with decreasing priorities)

⇒ TRAP to RST 5.5 are vectored interrupts → routine is serviced at a particular location. INTR is non-vectored interrupt. Whoever has generated the interrupted must provide the address of routine.

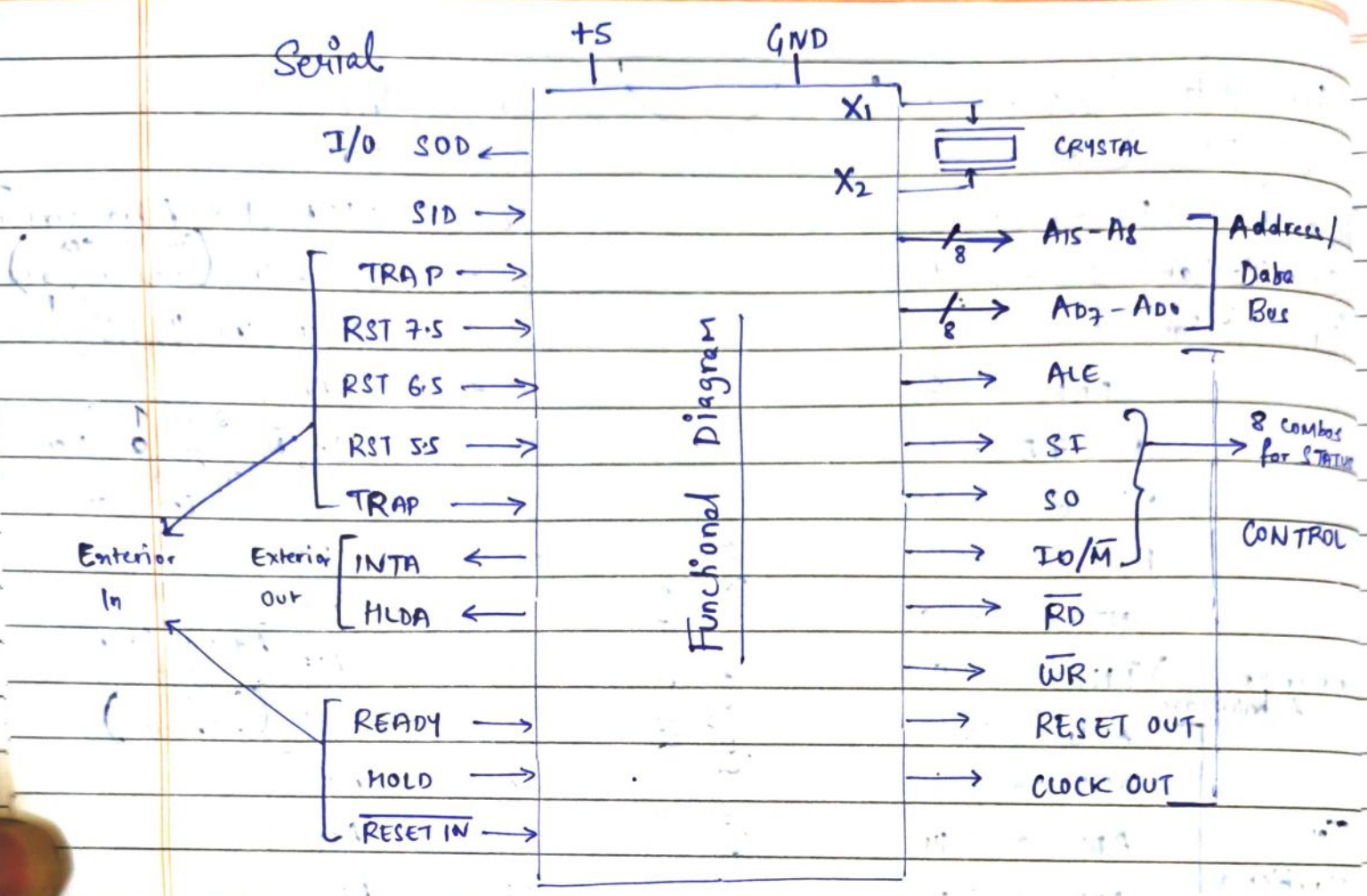
→ INTA → Interrupt Acknowledgement. (for INTR).

→ Code for vectored interrupt is written at a specific location.

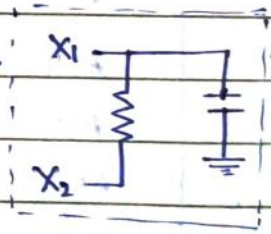
Accesses

o DMA (Direct Memory Access) → Memory of CPU. Requires 2 buses; addresses and data buses.

Op code (function)



Crystal is more favorable but can be replaced by →



→ RESET out Resets peripheral devs.

→ Ready signal helps CPU read data when high.

→ PC location must be known at power ON.

→ High Address → $A_{15}-A_8$

Low Address → AD_7-AD_0 (These are multiplexed).

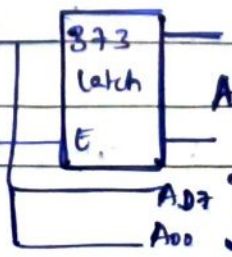
time

→ ALE signal high for every first clock of instruction. For every subsequent signal ALE ⇒ low.

When we need new addr. ALE-high → AD_7 AD_0

ALE

→ When ALE low AD_0-AD_7 can be used as Data Bus (and reduces no. of pins).



E-1

PAGE NO.
DATE / /

DOSBox

Z:\> mount n 'spacebar' /home/student/cdos. : mounting a folder

command = mount/unmount.

On Terminal

(1) mkdir dos1
cd dos1
ls -al

(2) pwd ⇒ /home/student/dos1

(3) cat > fqile
" sentence " Ctrl+D

(4) In DOSBox →
mount V /home/student/dos1 'V' is mounted.
V: → changes drive. (from Z to V.)
dir → shows directory of V.

(5) Ctrl+F9 to close DOSBox.
Ctrl+D to close Terminal. or disconnect from server.

(6) ssh m1439@172.18.2.10 → connect to server.
password 1qwer432

(7) ls ../files

(8) scp m1439@172.18.2.10:/home/Mb26/files/DEBUG85.EXE .
(secure copy) same password.
current destination
(if other destination we write path or ..)

PAGE NO. :
DATE : / /

(9) DEBUG8S in DOSBox to enter Macroassembler.
No need for file name just press 'ENTER'.
Write 'h' afterwards to open 'Commands of simulated Intel's 8085 CPU debug programme'.

(10) R→ display registers contents.

(11) cat ls (ls is binary).
↳ command is cat /bin/ls.

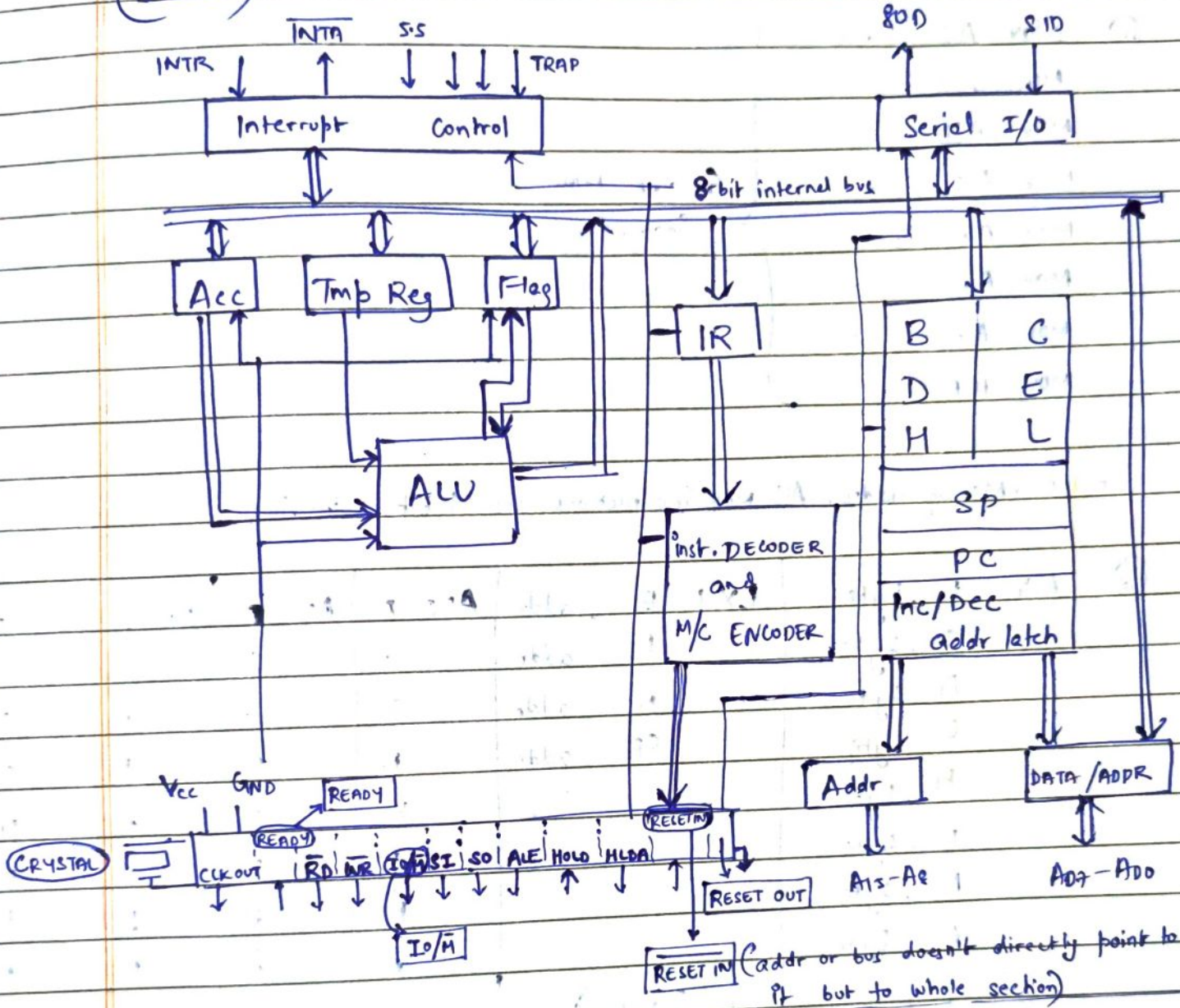
(12) .COM → executable type of file.

Refer mail attachments – 1 and 2 on 29th July

PAGE NO.:
DATE: 30 July

(3) FUNCTIONAL BLOCK DIAGRAM

(8085)



(addr or bus doesn't directly point to it but to whole section)

→ All instructions do not take same clock (CISC) unlike (RISC).

→ B, C, D, E, H, L are 8 bit registers.

→ SP and PC are 16 bit.

→ High Impedance Bus (for ~~data~~ addr bus ??)

INSTRUCTION SET

PAGE NO.:
DATE: / /

(1) MOV A,B ...
MOV A,C ...
MOV A,D ...
MOV A,E ...
MOV A,H ...
MOV A,L ...
MOV A,M ...
MOV A,A ...

64 such MOV instructions.

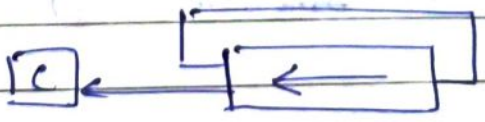
- M → Memory addrs. All MOV instructions are 1-bit instruction.

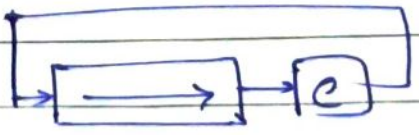
(2)

MVI B, byte	LXI B, addr	ADD B	ADC B	SUB B	SBB B
C, byte	D, addr	C	C	C	C
D, byte	H, addr	D	D	D	D
E, byte	SP, addr	E	E	E	E
H, byte		H	H	H	H
L, byte		L	L	L	L
M, byte		M	M	M	M
A, byte		A	A	A	A
2 byte	3 byte				
Move immediate	Load-extended immediate	Add into A	Add with carry into A	Sub from A	Sub with Borrow from A

(3)

RLC
RRC
RAL
RAR





(4) ADI, ACI, SUI, SBI, ANI, ORI, CPI.
(AND) (OR) (COMPARE)

PAGE NO.
DATE : / /

(3)

ANA B	XRA B	ORA B	CMP B
...	...	...	...
...	...	...	...

(AND) (XOR) (OR) (COMPARE)

Compare (CMP) ⇒

A <	C=1, Z=0
A =	C=0, Z=1
A >	C=0, Z=0

(4) 8085 [ISA → Instruction Set Architecture]

PAGE NO.:
DATE: 03 Aug

MOV (64 combinations)

ADD
SUB
ADC
SBB
} A is implicit/destination

ISA

ANA } logical AND
XRA } logical XOR
ORA } logical OR
CMP } compare

RLC | RAL } Rotate left/Right
RRC | RAR } with carry/without

MVI, byte
ADI, byte
ACI
SUI
SBI
ANI
XRI
ORI
CPI
} (Immediate)

LXI B, addr
D
H
SP (can be used to initialize SP)
} 3-byte instruction

INR } Increment
DCR } Decrement
} not ALU

INX } 16bit } INX B or D or H
DCX } DNX B or D or H

DAD B
D
H
SP
} Double Add in HL
e.g. HL + BC

LDA, addr (contents in A = content of addr)
LDAX, B (A = content in BC pointing (or DE) to Memory)
D
STA, addr (store value of A in MEM)
STAX, B
D

LHLD addr
SHLD addr

- Notes → (i) MOV M, M does not exists.
  (ii) Carry means error.
  (iii) First subtraction is SUB, all subsequent are SBB. (Multi-byte sub)
  (iv) logical operations are faster.
  (v) Compare → subtraction without storing result. Compare with A.

(5) ISA Continued...

SPHL [SP ← HL value]

PCHL [PC ← HL value]

XTHL

JMP addr → doesn't push RET address.

Jcc with
- JNZ Z=0
- JZ Z=1
- JNC C=0
- JC C=1
- JPO P=1
- JPE P=0
- JP S=0
- JM S=1

CALL addr (3 byte)

CNZ
CCC → pushes return address into stack
CPM

RET → pops return address from stack.

RNZ
RCC

PUSH B
D
H
PSW
→ one byte.

pushes into memory pointed by 'SP'.
'SP' points to last full memory.
→ A + PSW

POP B
D
H
PSW

IN (8 bit port)
OUT (8 bit port)
to/from A. (256 ports).

RST n (n: 0–7).

→ Follow little endian. i.e B high in BC. A is higher byte.

- PUSH B →
  - SP ← SP-1
  - [SP] ← B
  - SP ← SP-1
  - [SP] ← C
- Memory still 1-byte.
- CC → condition.

PSW → | S | Z | - | A | - | P | - | C |

PAGE NO.
DATE :

→ We don't send .Exe files via E-mail due to security.
It may send a ~~bug~~/useful program which we can't differentiate.

- scp mXXXX@172.18.2.10:/home/mp26/files/lab-sheet-1 .
- vi lab-sheet-1

{ ↑ ← → ↓ esc p :q! }

(12 characters)

→ filenames in older operating systems → 8 characters + 1 character for (., dot) and +3 character for extension.

→ If we don't rename file from txt to exe, prg says relevant prg not found.

- DEBUG85
  ↵ ENTER
  - (ready for prompt) 'h' for help.

→ address → 4 nibbles → Eg. 10 X 0010 ✓
byte → 2 nibbles . Eg. 05 ✓ 5 X

→ Commands:
a → assemble → take ↵ →
u → unassemble →
d → display memory
e → modify memory content
p → Init PC
r → Modify the register
R → display the register

PAGE NO.
DATE : / /

- a0100 ↵
  0100 : LXI H,0200
  0103 : MVI B,44
  0105 : MOV A,B
  0106 : MOV M,A
  0107 : HLT
  0108 : ↵

- u0100 ↵

→ low byte → low memory.

→ no address → last used address is continued. Eg. – d ↵

- d0100
  h0100

→ S → high byte of stack.
s → low byte of stack.

→ r → current register modification
[ ] → spacebar → next address.

→ e → write address
[ ] → spacebar → next address.

PAGE NO.:
DATE: 6 / Aug /

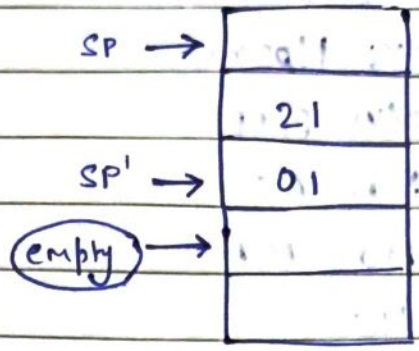
⑥ RST n

↓

n 0 → 7

2100	RST 3
2101	

(PUSH PC)



- STEPS →

1) PCH → SP-1
2) PCL → SP-2
3) SP ← SP-2
4) PC = 0018 ~~(3×4=12d)~~

- Code should ideally begin from $(0040)_h$ outside of RST range.

→ RST 4.5 → TRAP.

- INTR and $\overline{INTA}$ → Acknowledge.
  ↳ not for immediate action; interrupt.
  (level trigger).

Min
1 Byte
2 Byte
3 Byte
Max
} → Opcode Fetch $\overline{INTA}$ (needed) works until all Opcode Fetching is complete.

→ This is different from RST which 1) ~~does not need~~ further code and takes immediate action.
2) address fixed; INTR address is freely chosen.
3) deals with only 1 Byte Opcode.

- LHLD addr — HL ← [ ]
  SHLD addr — HL → [ ]

SPHL	SP ← HL	
PCHL	PC ← HL	(Kind of Jmp).
XCHG	HL ↔ DE	
XTHL	SP[ ] ↔ HL	

↳ exchange (top of stack) HL.

CMC	$\overline{Cy}$	(Complement carry)
STC	Cy = 1	(Carry set)
CMA	$\overline{A}$	(Complement caccumulator)
EI	Enable Interrupt	
DI	Disable interrupt	
NOP		
RIM	Reset interrupt Mask	{ can also be used to send/receive ~~interrupt~~ one bit at a time }
SIM	Set interrupt Mask	
HLT		

{ESP-32 dev code} breadboard Radio market

(7)

2s Complement Arithmetic

BCD (Binary Coded Decimal)

Counters / loop / Delay

(i) → in 8085 if no. is signed → 7E (+ve) + 1 = 80 (-ve).
for 2s complement retain 1st '1' and then flip subsequent bit.

$5 + (-4) = 5 + FB = 101$ → carry.

-4 → FC

earlier borrow gets cancelled from later carry

borrow

$-5 + 4 = FF$ ⟶ but no carry.

↳ -01.

(ii) byte → (upper nibble, lower nibble)

packed BCD ⟶ each nibble = 1 decimal no.
unpacked BCD ⟶ each byte = 1 decimal no.

Rule:

① After BCD addition (i) if lower nibble > 9 or AC = 1
add → 6d in result (lower)

(Auxillary carry) ← from lower to upper nibble.

~~②~~ (ii) if upper nibble > 9 or Cy = 1
add 6d in upper nibble.

PAGE NO.:
DATE: 11 Aug

⑧

⇒ I/O → serial → SIM, RIM (Interrupt Mask Set/Read).
→ parallel → IN/OUT

- Serial I/O { d_7 d_6 d_5 ... d_0 (related to interrupt) }
  ↑ ↑ data SI/O → Enable = 1 or disable = 0

SIM ↔ ~~read~~ write d_7 → out
RIM → read ~~d_7~~ ← in

- MVI A, 0XC0 ①
  SIM ← A → PIN 4 = 1.
  CALL DELAY 1sec
  (A moves to SIM, SIM pushes pin 4 out, and sets it high.
  delays 1 second.

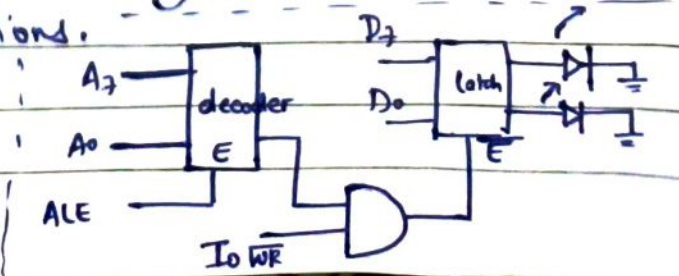
→ Continuation (serial I/O).

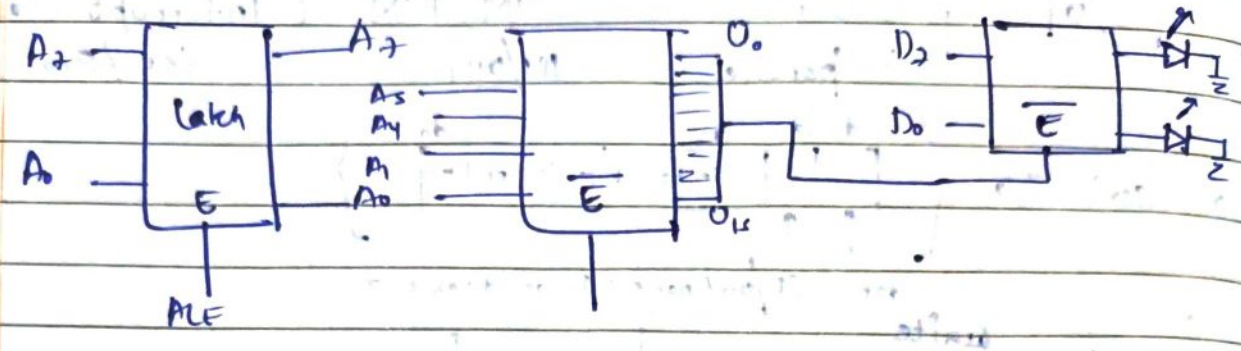
- XRI , 80
  ~~CALL~~ SIM
  CALL DELAY 1 SEC
  JMP ①.

- Parallel I/O → IN (from where) port → A ←
  OUT port (to where) ← A →

→ ALE appears in T1 irrespective of machine cycle.
e.g. IN FC ; the FC address is reduntanly copied to both address bus; A_7–A_0 and A_{15}–A_8. → low address is copied in high address.

- 256 Input port locations.
  256 Output port locations. } IO/M pin.
  2^{16} = or 65536 Memory locations.
  Total → 66048 combinations.




A7
A0
Latch
E
ALE
A7
A5
A4
A1
A0
E
O0
O15
D2
D0
E

E-3 LS-2 (Maybe leave 13 and 14).

→ Single and multi-byte add. (separation of carry).
→ Quotient and Remainder from multiple operations.
→ DAA = Decimal Adjust Accumulator

Code
```
-a 40
MVI A, 05
MVI B, 04
ADD B
HLT
```

→ How to edit program?
(i) -a 0044 ...
(ii) -m (move or copy). ; -m 0045 0046 0080
(iii) -e 0044 → modify.

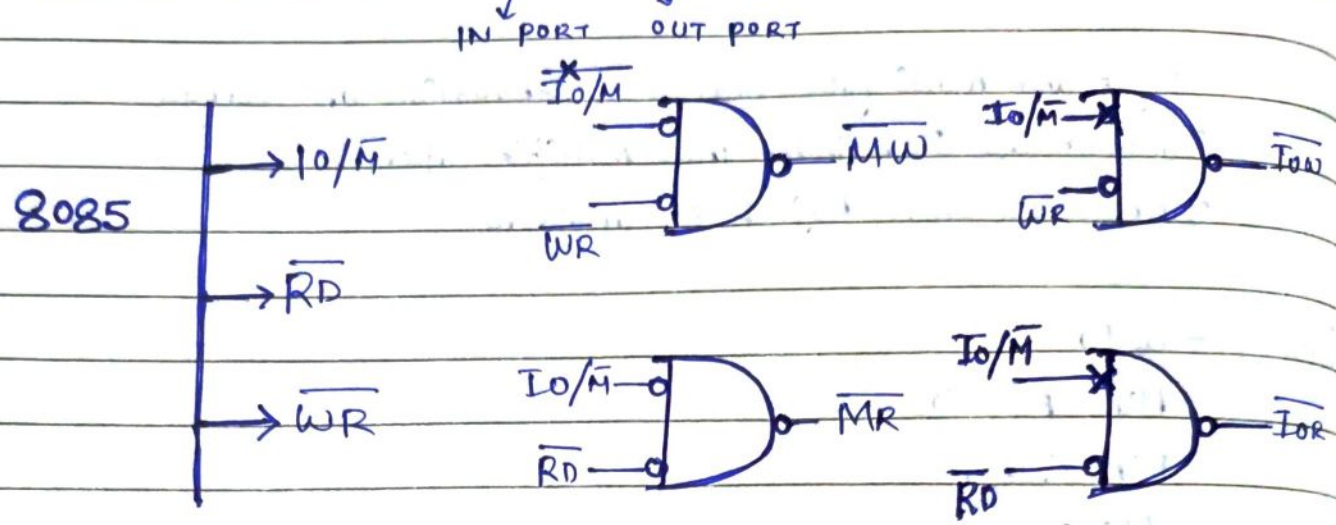
→ How to Save?
→ lab sheet
-s 1439201.asm
→ question
→ 0040 (starting address).
→ length of code.

(an .hex) ese iq:

to load? → l 1439201.asm

PAGE NO.
DATE 13 'Aug'

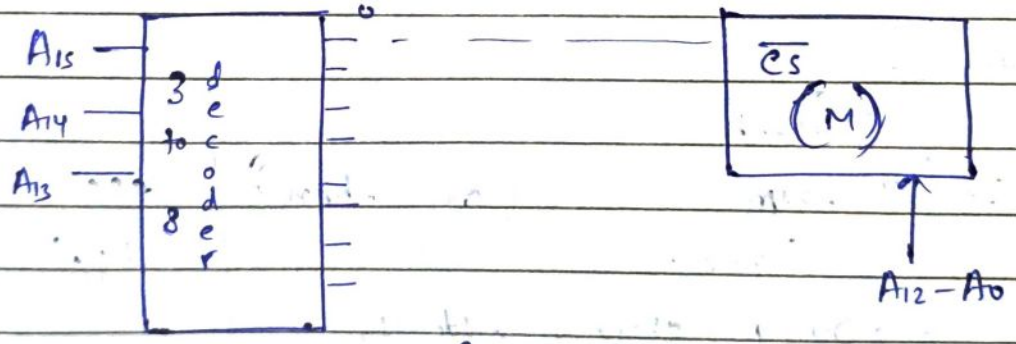
⑨ I/O → Parallel



→ interfacing

Address → Decoder → One of o/p of decoder.

→ asserted low decoder = asserted low CS.
asserted high decoder = asserted high CS.



(example).

→ 16 lines = 16 ports if decoded.
64k ports if encoded.

but port no. → 8 bit → 256 ports. for each IN and OUT.

- when we IN BC; the port no. 3C is copied from lower address to higher address. (no specific meaning).

→ once IN 3C is executed, 3C in High Address remains and low address remains free to perform RD/WR operation using ALE (Address Latch Enable).

(10) Loop/Wait/Delay

INR, DCR → Flag affected except Cy. (8-bit)
INX, DCX → Flag None (16-bit)

No. of times of loop ⇒ count (no.) — 8-bit and 16-bit

→ For down counter take -ve nos. (eg. -4 or FC);
" up counter " +ve " (eg. 4).

decided by 0-flag (Z-flag)

- How NX decided when loop is to be stopped?

(A) ← Move on register in Accumulator.

B	C

(eg. B in A).

```
MOV A,B
ORA C
JZ
```

Then 'OR' with another; if Z=1, Jump.

→ Initializing of counter can be done using : MVI C, byte.

```
      MVI C, 00
L1    [ execution ]      } Do until
test  { DCR (FF)
      { JNZ L1           }
```

```
      Init Counter
L1:   test {             } Do while
      JZ L2
      [ execution ]
      JMP L1
L2:   ----
```

- Delay/Wait

f = 2 MHz
t = 0.5 μs

(Code - Example)

```
      LXI B, number   - 10
L1 ←  DCX B           - 6
      MOV A,B         - 4
      ORA C           - 4
      JNZ L1          - 10/7
                        (T) (F)
```

OP Fetch
e.g. MOV → 4T (clocks).
CALL → 18T
PUSH → 12T
RET → 12T

Delay = 10 + N[6+4+4+10] - 3 → no. of clocks

→ Nested Delay

```
LXI B, N1
DCX B
LXI D, N2
DCX D
MOV A, D
ORA E
JNZ
MOV A, B
ORA C
JNZ
```

D1

Delay =

$10 + N_1[D_1 + 6 + 4 + 4 + 10] - 3$

$N_2(10) + (6 + 4 + 4 + 10) - 3$

$N_2 = 3 \Rightarrow 10 + 3(24) - 3 = 79$

$N_1 = 2 \Rightarrow 10 + 2[79 + 24] - 3$

$\Rightarrow$ 213 → clocks

If-Else

PAGE NO:
DATE: / /

(11) Compare

→ CMP R ← Register (B, C, D, E, H, L)

CPI byte

Cy	Z	Comp.
1	0	A<
0	1	A=
0	0	A>

Compares by copying the value in A and subtracting from accumulator, not saving result in the process keeping the flags.

o interrupt →

EI, DI, SIM, RIM ⟶ Read Mask (Attention).

↳ Set Mask (ignore).

(1) (0) → if we don't want to get interrupt while serving a interrupt.

↳ interrupt Flip Flop → (INT FF)

(for further interrup)

→ INT = TRAP + INT FF (RST 7.5 . M7.5 + RST 6.5 . M6.5 + RST 5.5 . M5.5).

TRAP → Level Edge

RST 7.5 → Edge

RST 6.5, RST 5.5 → Levels.

(M7.5 (eg.) → Masked/Unmasked).

? Doubt

Edge :- interrupt once and go away. (records presence in FF).

Level :- Keep interrupting.

→ SIM → (Works in Accumulator).

Bits in A.

SOD	SOE	X	R~~ST~~ 7.5	MSE	M7.5	M6.5	M5.5

SOD → o/p data remains in FF.

X → No bit

R7.5 → 1 = Reset 7.5 FF

MSE → 1 = masking, 0 = unmask

M7.5, M6.5, M5.5 → 0 — unmask (enable), 1 — mask (disable)

decides whether next 3 bits can interrupt you or not.

→ RIM →

Bits in A.

SID	I7.5	I6.5	I5.5	IE	M7.5	M6.5	M5.5

I7.5, I6.5, I5.5 → pending interrupt

IE → status of INTFF

(+1)

PAGE NO.
DATE : / /

0

1. TOS → pointed by 'SP'

(2) at RIM

5.5 Inte.

```
0 11  1  0 111
0 00  1  0 000   (and)
-----------------
0 00  1  0 0 00
```

JNZ IBR of 5.5

(3) → 'G' → 'go' in Dos bcoz commands keep running. 'T' → execute

(4) defaults: interrupts are masked

PAGE NO.:
DATE: 20/08/26

(12) # I/O interfacing (8085)
[8086]

→ Assembly Prg Format Sheets → {Rough official Example}

arbitrary example

Address	hex	Label	Opcode	Operand	Comments
0040	21		LXI	SP, 1000	init. SP to 0x1000.
0041	00		LXI	H, 0200	init. M pointer to (0200)h
0042	10	L1	MVI	A, 44	" "
0043	22		ADD	A	" "
0044	00		JMP	xxxx	" "
0045	02		HLT		" "

follow alignment

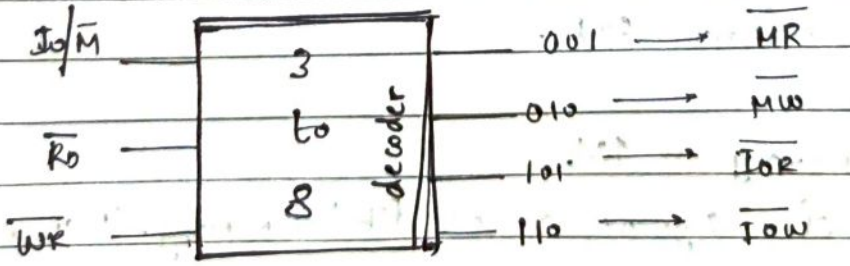
HLT hex code → (0076)h or (0x76).

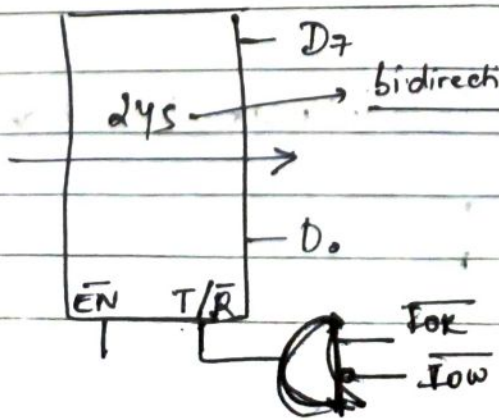
→ Little Endian: low byte low adress.

→ CR → Return to home of line. LF → Line Feed. {CR → 0x0A, LF → 0x0D}

- I/O instructions are 2 byte. (port No = 8 bits) ⇒ 2 machine cycles for Fetch.

→ Fetch of I/O is done from memory then IO/M becomes high to do i/p and o/p. (A_7-A_0 is copied into A_{15}-A_8).



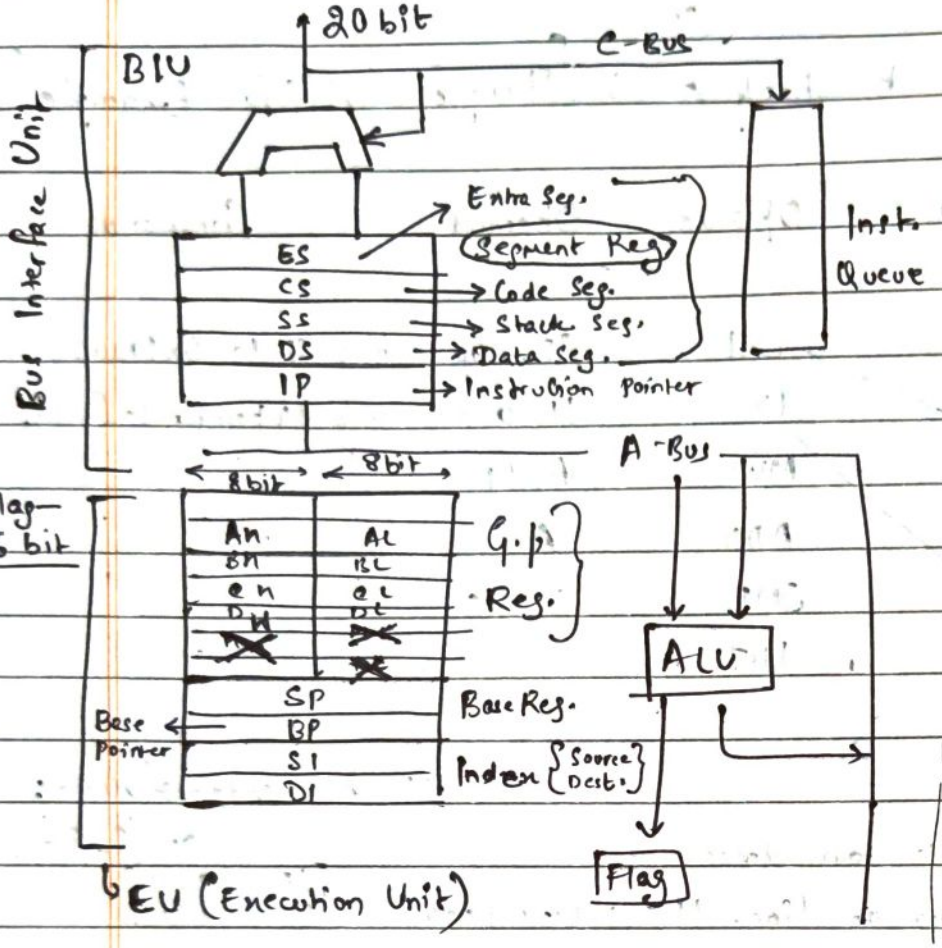


bidirectional buffer (245)

(T → Transfer, R → Receive.)

244 → Unidirection so only one dirn

(13) 8086 Microprocessor



- Double capacity than 8085.
- 16 bit - Flag →

not utilized.

0000 O_F D_F I_F T_F

S_F Z_F A_F P_F C_F

- Address: A_{19} - A_0.
- All internal registers are 16 bit.
- AX → AH (8 bit), AL (8 bit) → BX, CX, DX

- ES and DS are very useful in string operations.
- 32-bit : EAX, 64-bit : RAX.
- 16-bit pointers are within the segment. (pointing).
  Segment size (max) = 64 k.
- 20-bit memory = 1M.
  16 max size segments can be fit in memory. 4 users using 4 segments. (Multiple Users).
- O_F → Overflow ; D_F → Direction; I_F → interrupt (instead of EI/DI).

→ Mapping → $[(CS \times 10)_H + IP]$ (16 bit + 16 bit → 20 bit)

Q. How to access Operands? (Addressing Mode).

→ Register to Register
Register to Memory
Immediate Byte/Word to Reg/Memory
Memory to Register

(memory address)
direct (write immediate addr.)
indirect Register holds addr.

No Displacement	Inden	Base	Inden + Base
with displacement 8-bit (byte) 16-bit (word)	inden + disp.	bas + disp	Inden + base + disp

(14) 8086

- Addressing mode, Queue → 6 bytes.

memory Reference	index	base	index + base
no displacement	[SI] [DI]	[BX]	[SI+BX], [SI+BP], [DI+BX], [DI+BP]
with displacement → 8-bit [SI + 8 bit disp]			
16-bit [SI + 16 bit disp]			

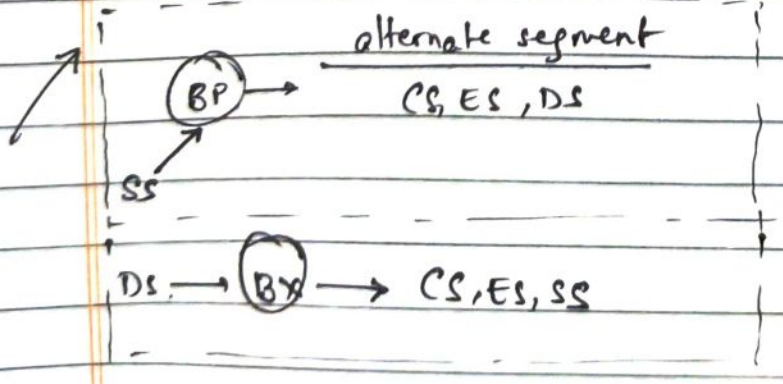
- BIU fetches instructions → in advance.

→ Non-sequential instructions ⇒ instructions in queue is meaningless and we have to flush it out / performance ↓.
Sequential instructions ⇒ meaningful instructions in queue.

→ pointer is indirect memory reference.

- default offset/seg

CS → IP
SS → SP, BP
DS → SI } if string op. is involved; source of string in DS
ES → DI } and destination in ES.



- 8-bits get converted into 16-bits ⇒ 80 changes to (FF80).

(6000 + ABCD) → offset < 16 bit (overflow) ⇒ 1 0BCD
[1010 1011 1100 1101]
6000 + ABCD

(15) Instruction Set of 8086

o Data Movement

1) MOV op1, op2 → R←Im; M←Im; R←R; R←M; M←R; R←S; S←M; M←S; S←R

AL, AX ... CH, CL ... — D ... B
AX ... B ... C ... D, SI, DI, SP, BP
→ Immediate memory

S → Segment R: CS, DS, ES, SS

XCHG op1, op2	LDS R, M	IN R, port
PUSH OP1 / POP OP1	LES R, M	OUT port, R
PUSHF	LEA R, M	XLAT → left as of now.
POPF		

Byte, Word, DWord

→ MOV Dest Soure ; Eg. if Source 20,

MOV Byte [M], 20 moves only (20) at one byte location.

MOV Word [M], 0020 moves (00) and (20) at two byte locations. (little endian)

→ PUSHF, POPF ⇒ used to push/pop flag.

o LDS BX, [1000] → load into DS. (Data segment) — see eg. below
(1000 → offset of memory.)

LES " " → load into ES (Entre Segment)

These read 4 bytes : lower 2 bytes initialize (BX). (higher 2 bytes initialize) (DS) the address of segment

o LEA R, M → Address of memory location is copied into register. (effective Address)

→ CS is initialized by Operating System.

→ IN AL, DX (read 8-bit data specified in DX). → specified port no. there.

→ IN AL, port no. or IN AX, port no.

Memory	
1000	AA
1001	BB
1002	CC
1003	DD

eg.
LDS BX, [1000]
DS : BX
DDCC : BBAA

8086 Instructions

PAGE NO.:
DATE: / /

Data Movement:

MOV, LDS, LES, LEA, XCHG, PUSH, POP,
PUSHF, POPF, LAHF, SAHF, IN, OUT, XLAT

- LAHF → load flag into AH register (lower byte).
- SAHF → save AH as flag. (lower ~~flag~~ byte).
- IN/OUT → can be byte/word width. 8085 was only byte width.
- XLAT (Translate) → to a Byte
  AL ← [BX + AL]
  if 'A' is written at address.
  AL ← 41

Arithmetic

Add op1, op2
ADC, SUB, SBB, CMP
OR, AND, XOR, TEST
INC, op1,
DEC, NEG, NOT
MUL op1,
IMUL
DIV op1,
IDIV

if Cy, O_F → set (16-bit result)
Cy, O_F reset (8-bit result)
for 8-bit MUL

DIV

dividend > divisor	if not divide by zero error
16 bit > 8 bit	
32 bit > 16 bit	
(dividend) AX	(dividend) DX:AX
AL = Q	AX = Q
AH = R	DX = R

- We need an encryption/decryption table to translate using XLAT.
- CMP DX, BX ⇒
  ADD, SBB, ADC, & SUB → any register.

	C	Z	S
=	0	1	0
<	1	0	1
>	0	0	0

→ Test → logical AND but doesn't affect destination (only flags).

→ INC, DEC → same as INR, DCR. (in 8085).
NEG → 2s complement, NOT → 1s complement.

Note → used only with DIV
CBW → byte to word
CWD → word to double
↓
copy MSB throughout.

→ IMUL, IDIV → signed/integer MUL or DIV.
one op is implicit and other is explicit. } sets C_F and O_F flags.
↳ (AL, AX)
16 bit product → AX. 32 bit product → DX:AX.
(if C_F and O_F are set that means it is a meaningful result in AH.)

(E-5) # DEBUG 86.

→ download : debug86.exe
in debug86 we use '?' for help.

→ No interrupt concept.

o Instructions →

A (assemble)
U (unassemble)
E (enter/edit) D → (Dump/display)
T (trace)
R (Register)

→ first instruction is offset at 0100 as 256 locations are reserved for i/o.

NV →	No overflow	if wrong flag written (buffer error) and certain flags are not visible when using 'R'.
UP →	Directional	
EI →	Interrupt	
PL →	Positive/Plus	(MN → Negative/Minus) ↳ NG
NZ →	No Zero	
NA →	No Auxilliary	
PO →	Parity	
NC →	No carry	

→ RAX to change Register AX.
→ RF for flags.

→ E with addr. (E 200)
→ E 1000:1234 (segment value 1000, offset 1234)
give values after giving enough

→ **ASCII code**

20 → byte space
41 → A
30 → 0
61 → a

} 64 printable characters from 21 → 80 (maybe) → 96 ?

→
```
MOV AX, 1245
MOV BX, 6339
ADD AX, BX
INT 20
```
} example code (1)

→ code segment

→ T=100 (single step execution. from addr 100).
T100 (100 instructions executed from 100).

→ example code (2)

```
MOV BX, 300
MOV [BX], 7E
```

(17) ASCII and BSD

→ AAA → Asai Adjust After Addition → AL = AL & 0F
if AL > 9 or AC = 1
AL = AL + 6; AH = AH + 1
CY = AC = 1
AL = AL and 0F (&)

AAS " " " Subtraction
AAM " " " Multiplication (hex to ASCII)
AAD → " " Division (ASCII to hex)
DAA → same as 8085
DAS → same base but subtraction.

AAM → Multiplication (hex to ASCII) → AH = AL MOD 10d
AL = AL / 10d

AAD → before Division (ASCII to hex) → AL = AH × 10d + AL
AH = 0

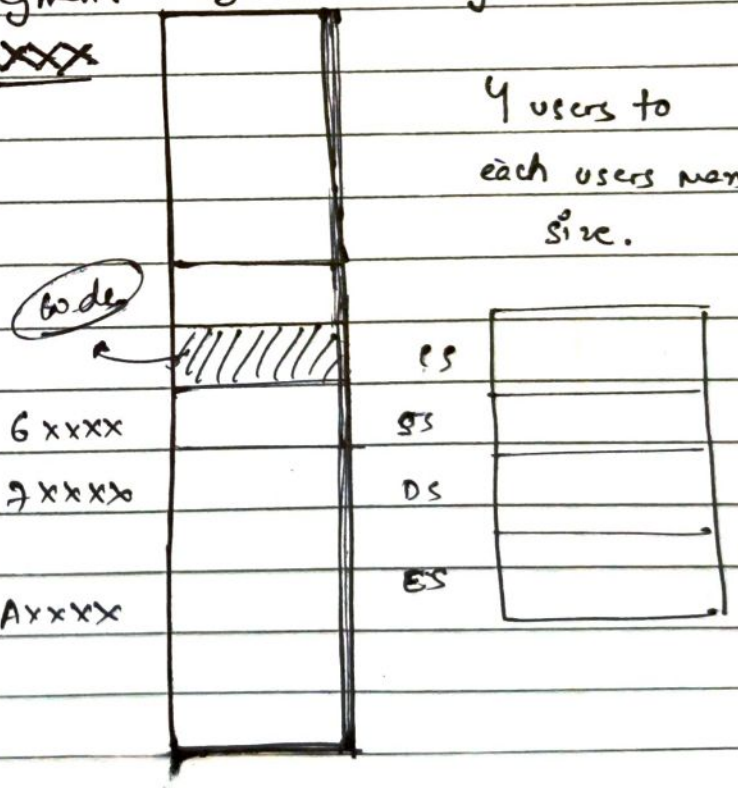
o Segments

Code segment Byte 1MI Byte

XXXXX

CS →
SS →
DS →
ES →

4 users to each users max's size.



If offset
SI
DI
BX
BP
SP
→ combination
(Segments)

Theoretical Calculation → Code segments are there to calculate physical address.
Segment Reg ⇒ 16 bit
64 K each no. of segments for each CS, SS, DS, ES.
64K × 4 = 2^{18}. (physical addresses).
size of IP = 16 bits = 2^{16} ∴ total memory = $2^{18} \times 2^{16} = \boxed{2^{34}} = \boxed{16\ GiB}$

→ No. of usual users (each requires all segments at least once)
→ 4

→ Divide each segment into CS + DS + ES + SS
∴ 16 distinct segments for each user ∴ 16 users.
(We have divided each physical segment into 4 different segments to cater for each user).
↳ Initially each user used 4 segments
now it uses only 1 segment.

→ if minimum size of each segment i.e $(10)_h$
then each user uses $(40)_h \rightarrow 2^6$
∴ 2^{14} users. $= 2^{20}/2^6 =$ 16k users

PAGE NO.:
DATE: / /

Flow Control

- Conditional JMP → short address (label) [JMPs are relative in 8086]
  - JO; JNO (overflow)
  - JS; JNS (sign)
  - same { JZ; JNZ / (JE); (JNE) } (equal or ZF set) → used alongside CMP
  - same { JPO; JPE / (JNP); (JP) } (Parity)
  - JC; JNC (Carry)

  range of JMP → −128d to 127d
  ↓ IP changes after fetching.

- repeat (LOOP)

LOOP	CX = CX−1; jump to label (addr) (if CX>0)
LOOPE / LOOPZ	CX = CX−1; jump to label if (CX>0 and ZF=1).
LOOPNE / LOOPNZ	(same but ZF=0).

→ REP and LOOP differ where REP first checks condition then decrements. LOOP first decrements then checks 'CX'.

→ if CX=0; LOOP can go on for '64K−1' times. but there is short address limit. (−128d to 127d).
LOOP calculates relative address based on what we write as jump address.

- Unconditional JMP
  - JMP direct → direct memory (5 Byte) Far / (3 Byte) Near → 2 Byte → short (Near).
  - JMP indirect → { Reg. → Near (−32768 to 32767). / Memory → Far }

→ Referring to Reg (Code) is possible for every register.
Referring to memory: '[BX]' is only possible via (BX, SI, DI, BP.)

→ short/Near → intra segment, Far → inter segment

JMP NEAR [BX]
JMP DWORD [BX]
JMP FAR [BX+SI+10]
JMP WORD [BX]

E-6 Lab-Sheet-4

o N XXXX049.COM → Name

R BX → 00 ; R CX → total length (32) here

size specified by registers BX:CX

to save space BX=00

W write into storage

L show that many bytes (load into memo).
(read from storage)

use unassemble U.

eg. → -U213

CS:213 MOV AX, 4142

243 INT 20

245

[N R W] [N L]

→ specify the address.

8086 addressing is relative.

→ on starting DOSBox again →
e0100 to edit / clean code.
load then.

PAGE NO. :
DATE : / /

(20) Flow Control

- JMP
  - Unconditional
    - relative → IP ± disp.
      - direct → NEAR / SHORT } intra
    - absolute
      - direct → FAR (inter)
      - indirect → Reg → NEAR (intra)
      - indirect → Mem → FAR (inter)
  - ~~absolute~~
  - Conditional (relative) → short

Summary of JMP

- operation (opcode → machine code)

SHORT	EB —
NEAR	E9 — —
FAR	— — — —

(low byte low address → little endian).

(machine code)

indirect	FF —
byte offset	FF — —
word offset	FF — —
FAR	FF — — — —

8-memory references

Memory addressing

[BX]	[BX+SI]	these 8 can be identified by 3 bits.
[SI]	[BX+DI]	
[DI]	[BP+SI]	
[BP] (**)	[BP+DI]	

- CALL → unconditional always → always absolute
- RET

Summary

CALL
- direct → FAR
- direct → NEAR — (push)
- indirect → Reg → NEAR
- indirect → Mem → FAR

→ for inter segment: 4 bytes to push (CS:IP).

→ for intra segment: push 2 bytes (IP).

eg. CALL FAR [BX + SI + 1234]

memory will read 4 bytes as destination.

- RET n → can be 16 bit or 2 byte no.
  - ↳ n more bytes are popped.